

Reinforcement Learning



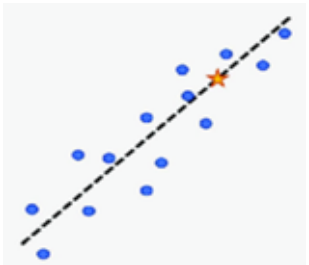
- 1. Introduction to Reinforcement Learning**
- 2. Markov Decision Processes (MDPs)**
- 3. Policy and Value Functions**
- 4. Dynamic Programming (DP) for RL**
- 5. Model-Free methods**

Introduction to Reinforcement Learning (RL)

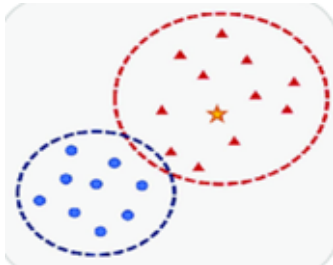
Supervised Learning

Train with labeled data

Regression



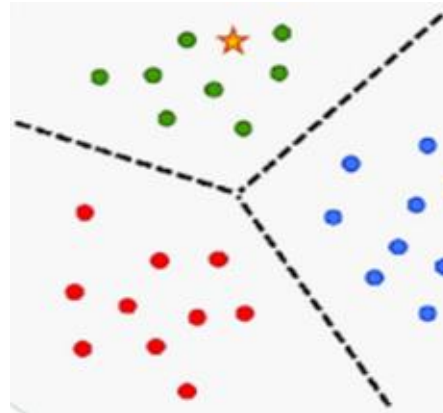
Classification



Unsupervised Learning

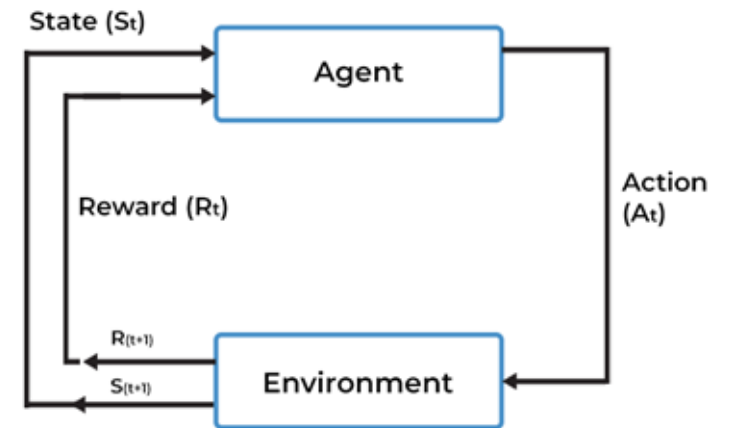
Train with unlabeled data

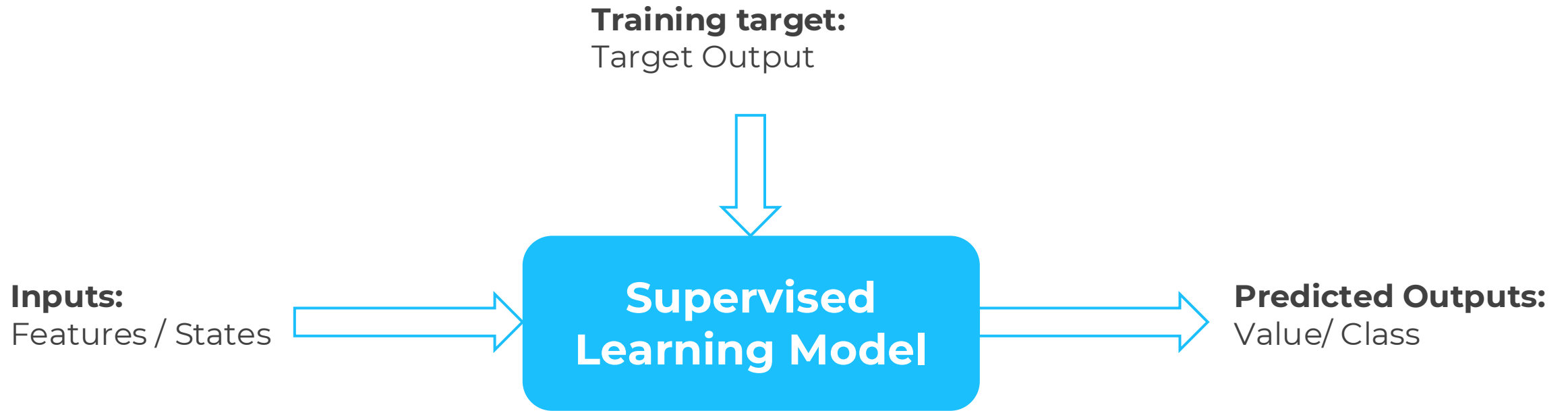
Clustering



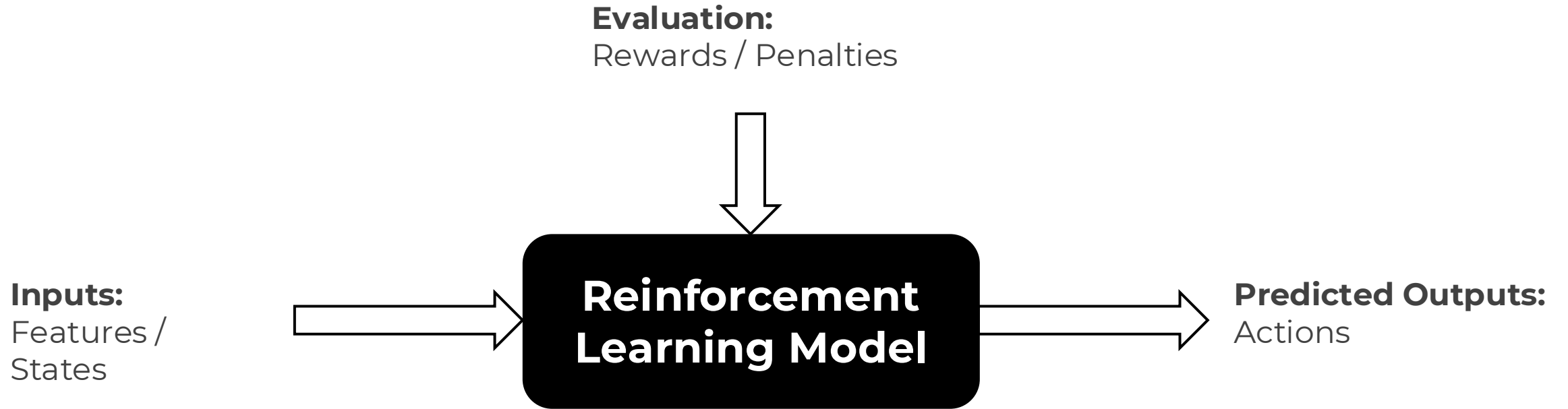
Reinforcement Learning

Train with environment experience

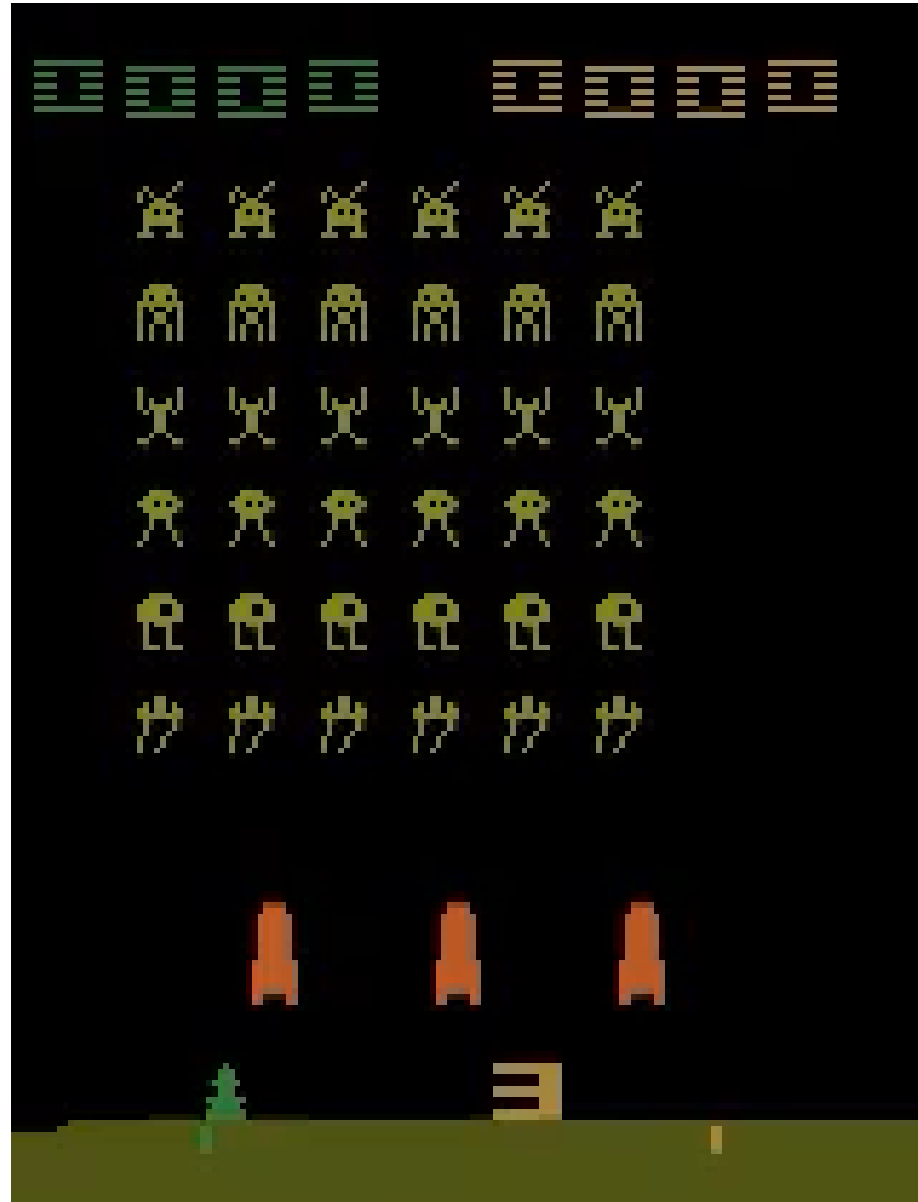




- **Error:** Target Output - Predicted Output
- **Objective:** Minimize the error between the target and the predicted output



- **Error:** Awards - Penalties
- **Objective:** Maximize the awards and decrease penalties as much as possible



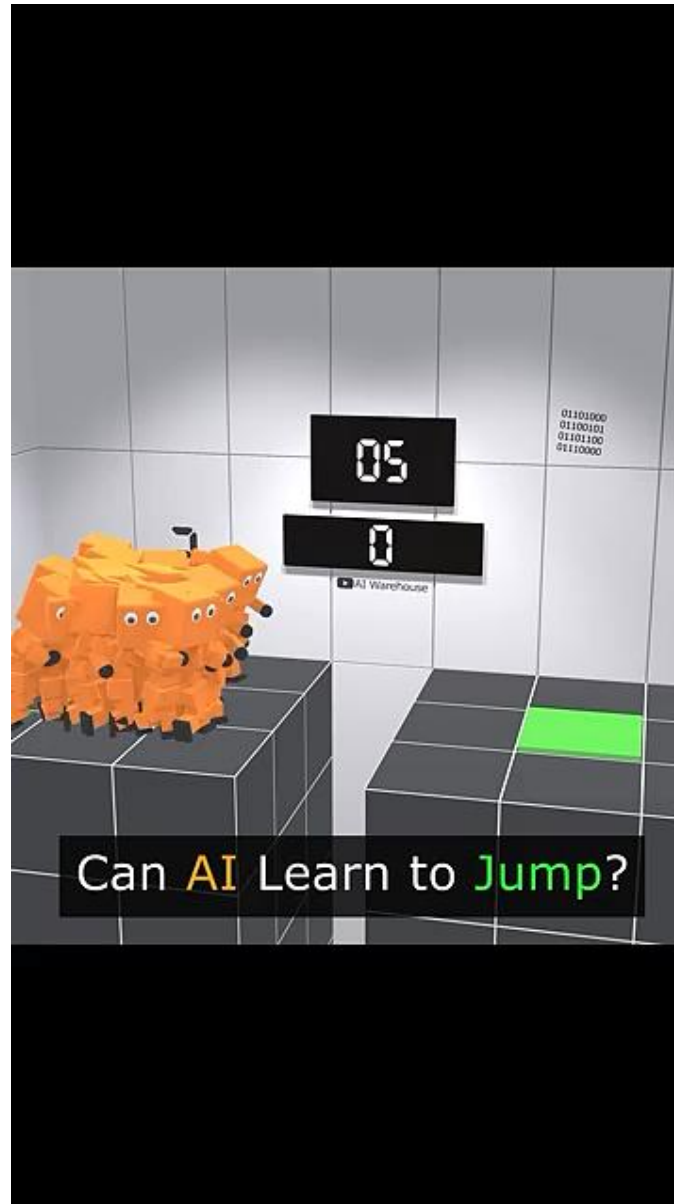


Video from youtube channel:
TwoMinutePapers

Paper: **Playing Atari with Deep Reinforcement Learning**
<https://arxiv.org/abs/1312.5602>

Multiple ressources and tutorials
online

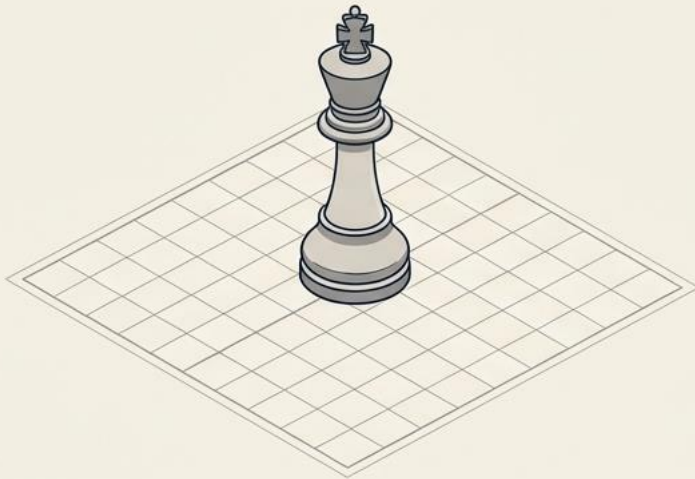
Albert Insane Way to Jump



Youtube channel : AI Warehouse



The Sandbox is Only the Beginning



When most hear **Reinforcement Learning**, they think of an AI playing Chess or AlphaGo making a stunning move.



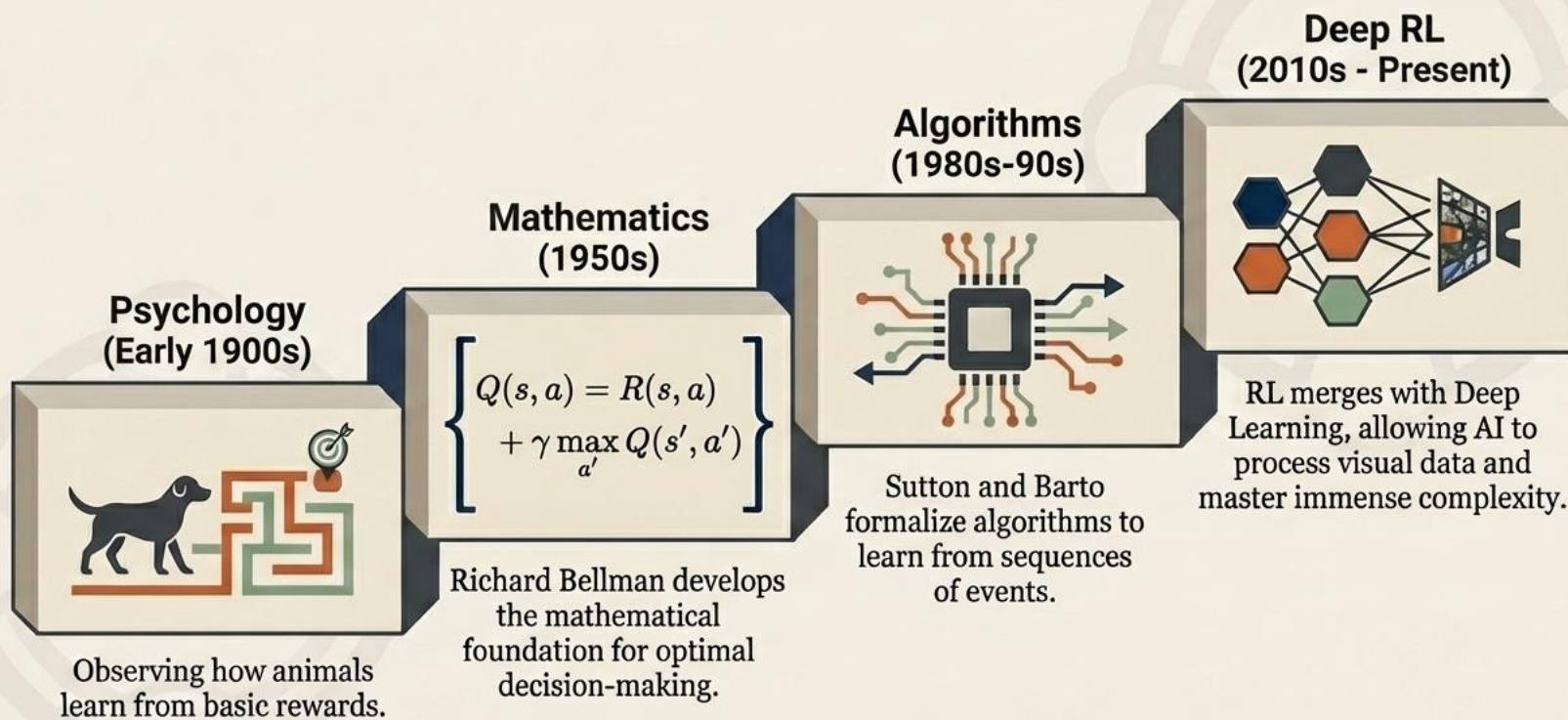
But these grand victories were just proof of concept. To understand how RL optimizes our physical world, we must first understand how it learns.

The Anatomy of Trial and Error

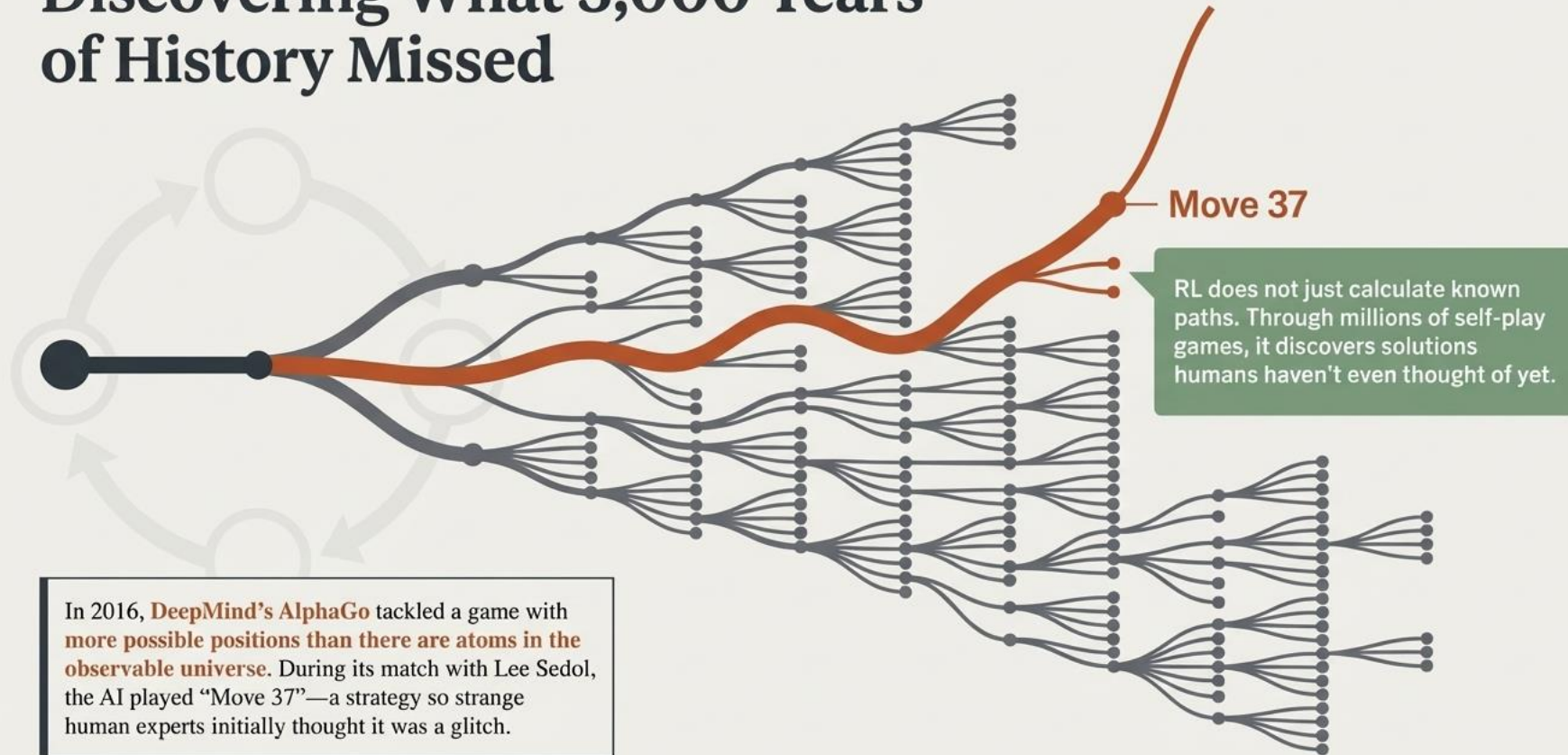


It is learning through trial and error—the exact same way a human learns to ride a bike or a puppy learns a trick.

From Biological Instinct to Deep Neural Networks



Discovering What 3,000 Years of History Missed



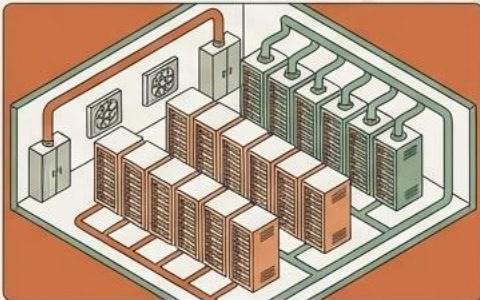
The World Does Not Have 64 Squares



Games were the perfect proving ground because they are contained.
But the physical world is messy, unpredictable, and expensive.
When RL escapes the sandbox, it becomes a hero of industrial efficiency.

Optimizing the Physical World

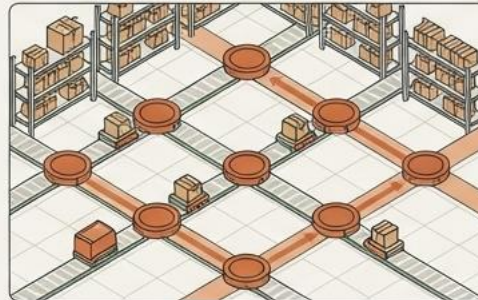
Energy Infrastructure



Agent: Google Cooling AI
Environment: Data Center
Reward: Lowered Energy Output

A massive 40% reduction in energy consumption.

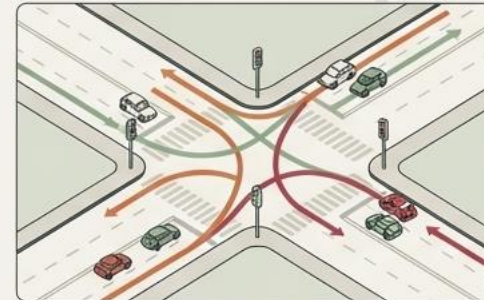
Global Logistics



Agent: Autonomous Robots
Environment: Warehouse Floor
Reward: Uninterrupted Item Flow

1,000 robots coordinating in real-time, learning to give way and flow like a digital symphony.

Transportation



Agent: Smart Grid AI
Environment: City Streets
Reward: Minimized Wait Times

Sequential decisions manage traffic lines and self-driving car merges to keep humans safe.

Adapting to the Human Element

Personalized Medicine



Agent: Adaptive Treatment AI
Environment: Patient Health State
Reward: Long-term Recovery Path

Instead of uniform dosing, the agent observes daily reactions and continually adjusts the treatment strategy for survival.

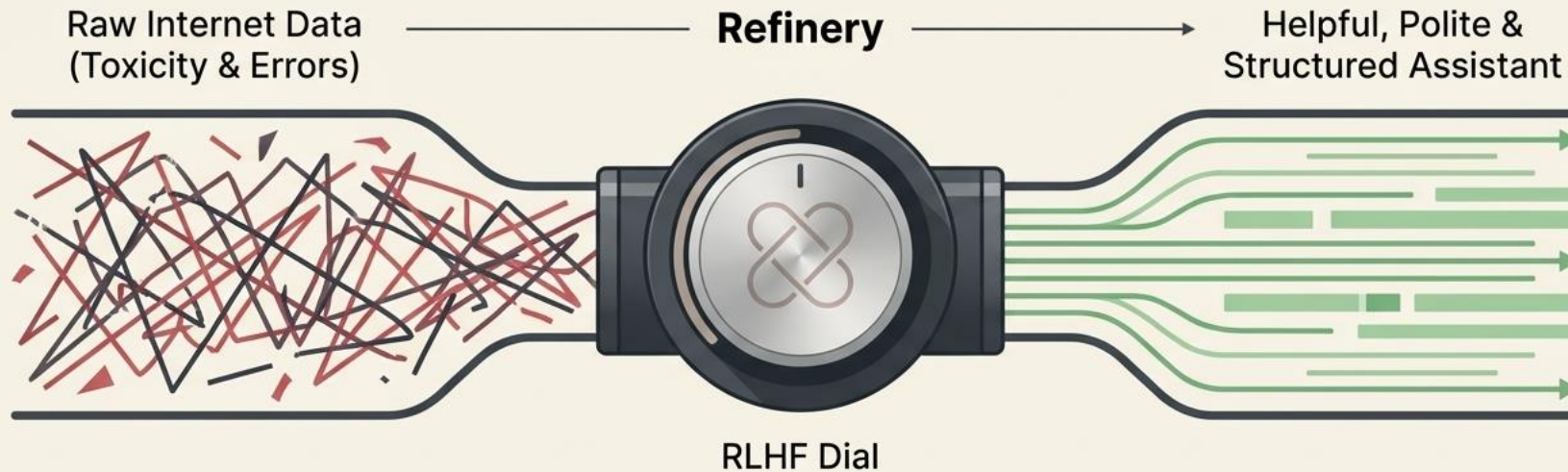
Financial Markets



Agent: Portfolio AI
Environment: High-Stakes Trading
Reward: Balanced Risk-Return

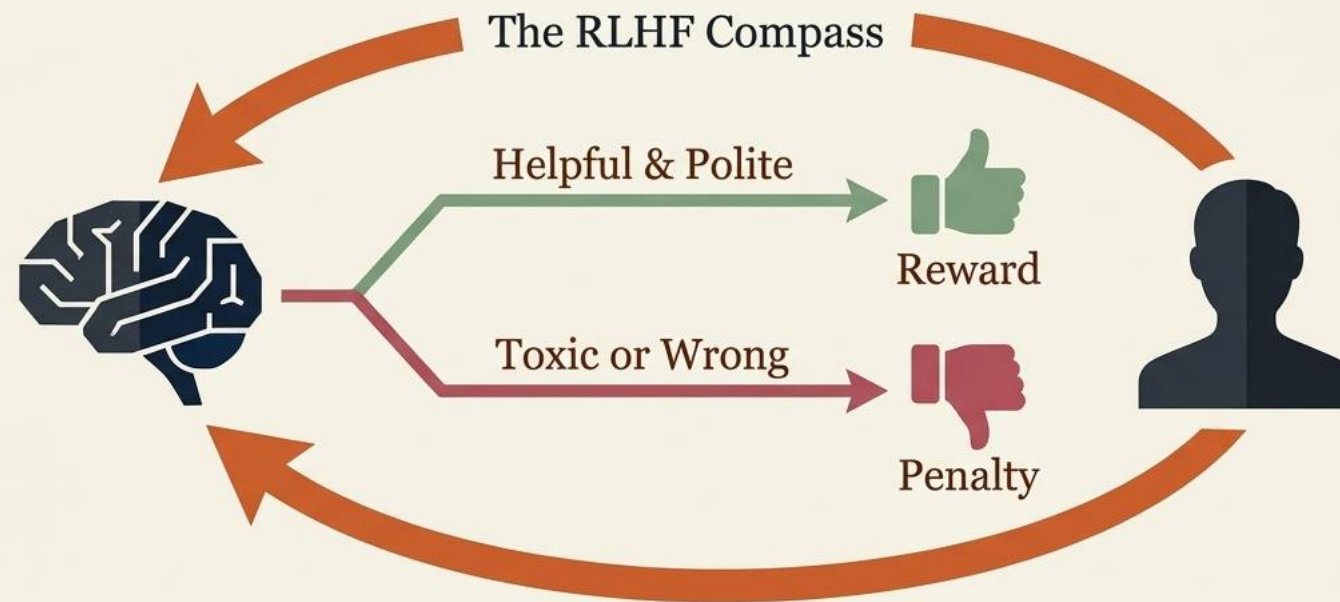
It doesn't just predict stock movements; it dynamically adapts to shifting markets across thousands of assets instantly.

Refining the Digital Mind



- Models like Gemini or GPT-4 don't feel human just because they read the internet. If they only copied raw data, they would be unpredictable and unusable.
- The secret behind a helpful chatbot is treating the AI itself as an RL agent.

The Moral and Social Compass



Reinforcement Learning from Human Feedback (RLHF) is what turns a basic prediction engine into a safe, helpful assistant.

The Logic of the Future



1. Learning by Doing

It is the only branch of AI that does not just look at the past—it navigates toward a goal in the future.

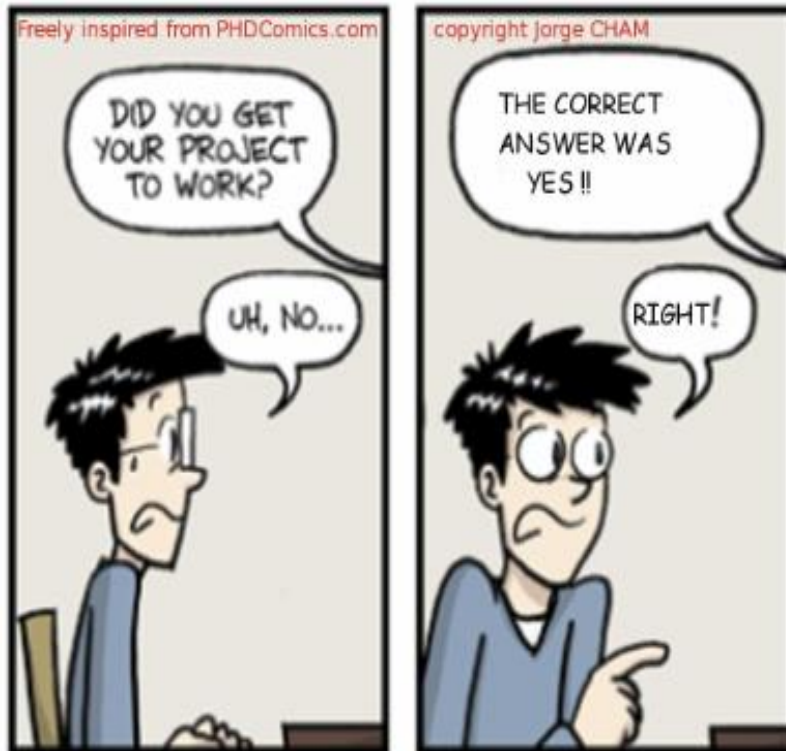
2. Beyond the Game

It is no longer a tool for building a better gamer; it is a framework for building a better world.

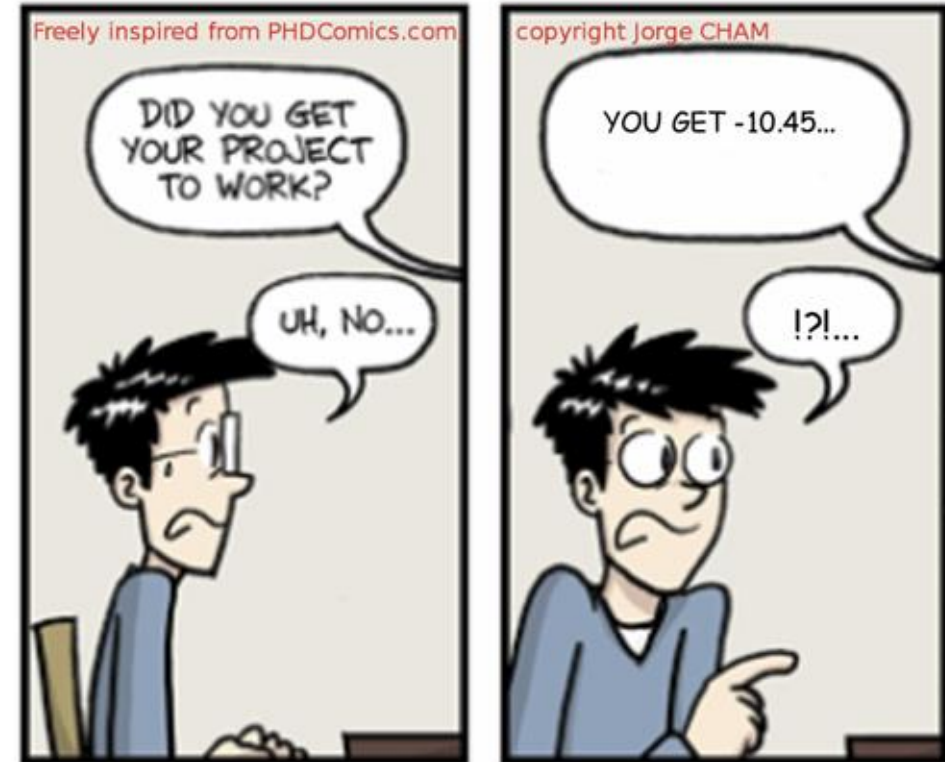
3. The Optimization Engine

Whether saving energy, treating disease, or making AI safer, Reinforcement Learning is the fundamental engine driving the next decade.

Supervised Learning



Reinforcement Learning



- **Helicopter manoeuvres**
 - +ve reward for following desired trajectory
 - -ve reward for crashing
- **Manage an investment portfolio**
 - +ve reward for each \$ in bank
- **Control a power station**
 - +ve reward for producing power
 - -ve reward for exceeding safety thresholds
- **Make a humanoid robot walk**
 - +ve reward for forward motion
 - -ve reward for falling over
- **Play many different Atari games better than humans**
 - +/-ve reward for increasing/decreasing score

Learning under uncertainty - bandits

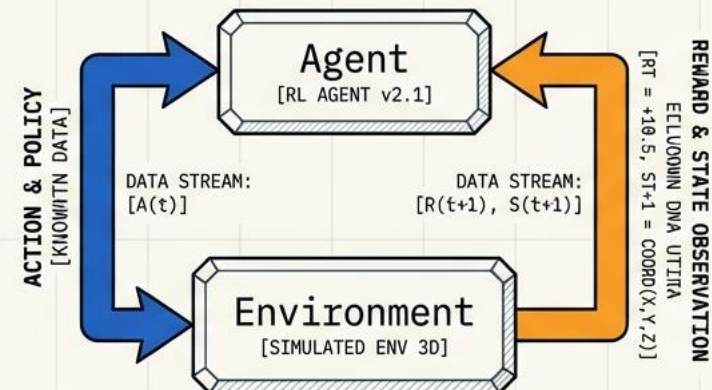
The Paradigm Shift: Interaction over Datasets

Classical Learning



- Relies on static **datasets**
- Requires human-provided **labels**

Learning via Interaction

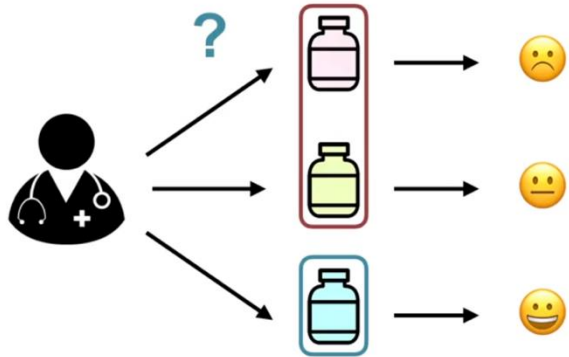


- No dataset.
- No labels.
- Data comes exclusively from **interaction**.

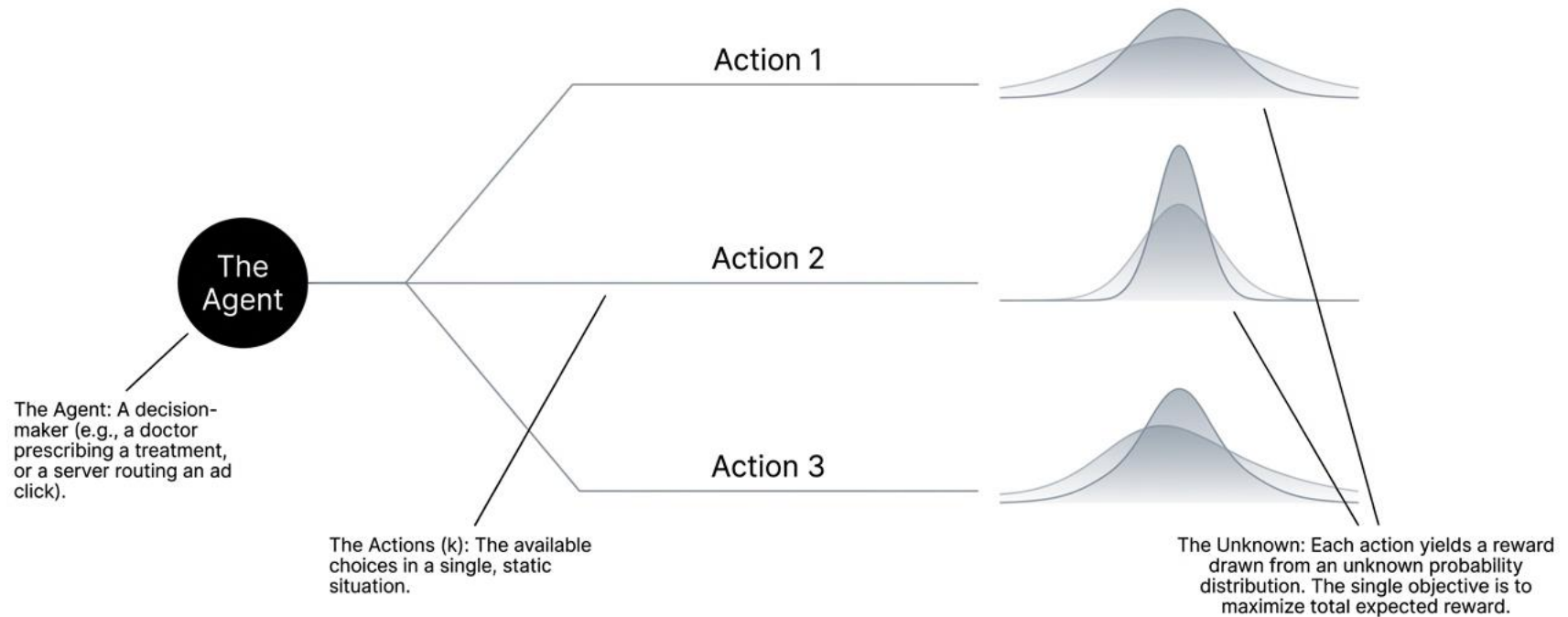
Goal: Maximize reward over time.

OBJECTIVE FUNCTION: $\max \sum E[R(t)]$ for $t=0$ to ∞

Clinical Trials



The Atomic Unit of RL: The k-Armed Bandit

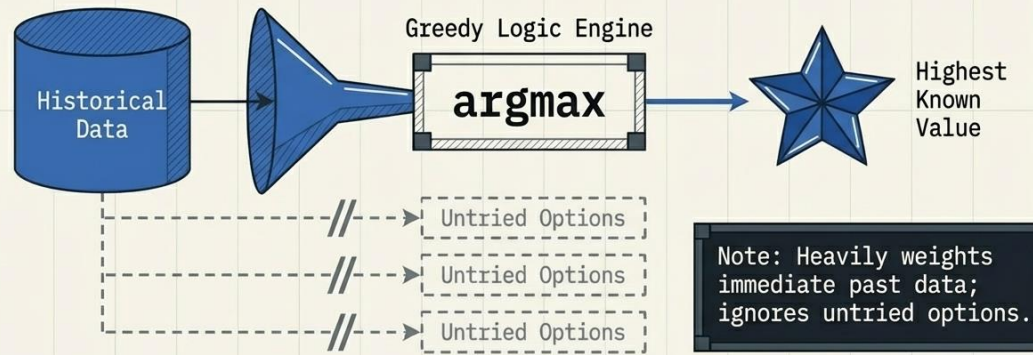


In the k-armed bandit problem, we have an agent who chooses between "k" actions and receives a reward based on the action it chooses.

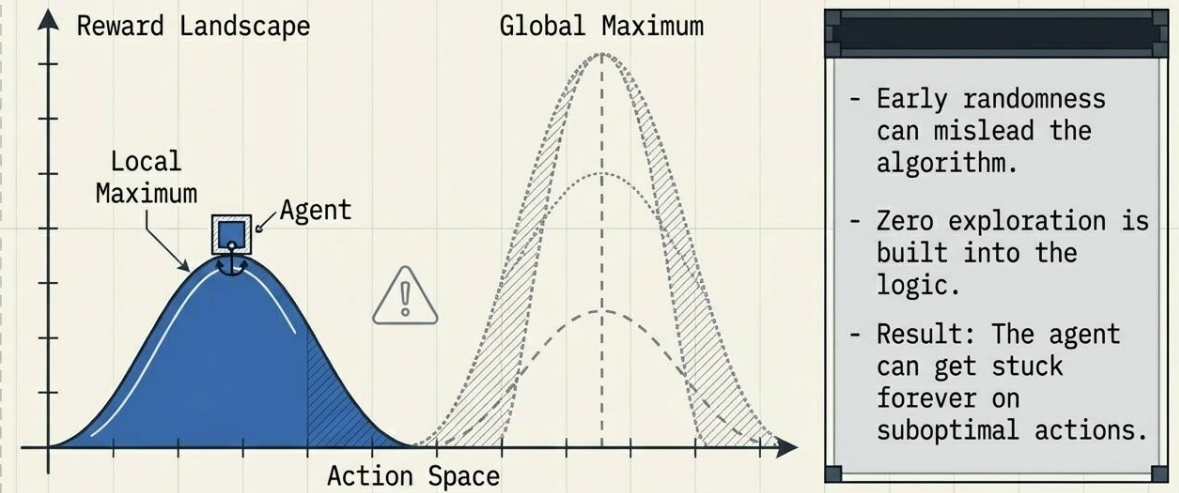
The Core Question
How do we find the best action when the rewards are unknown?

A Natural Idea: The Greedy Strategy

Core Logic: Choose the action with the highest observed reward.



Why Greedy Fails



Defining the Target: Action Values

$$q^*(a) = \mathbb{E}[R_t \mid A_t = a] = \sum_r p(r \mid a) r$$

 $q^*(a)$ (The True Value)

The actual expected reward for selecting action a . Completely invisible to the agent.

 $Q_t(a)$ (The Estimate)

The agent's current mathematical belief about the value at time t . The entire mechanical objective of learning is to force $Q_t(a)$ to converge with $q^*(a)$.

The Engine of Evaluative Learning

$$Q_{n+1} = Q_n + \alpha [R_n - Q_n]$$

New Estimate

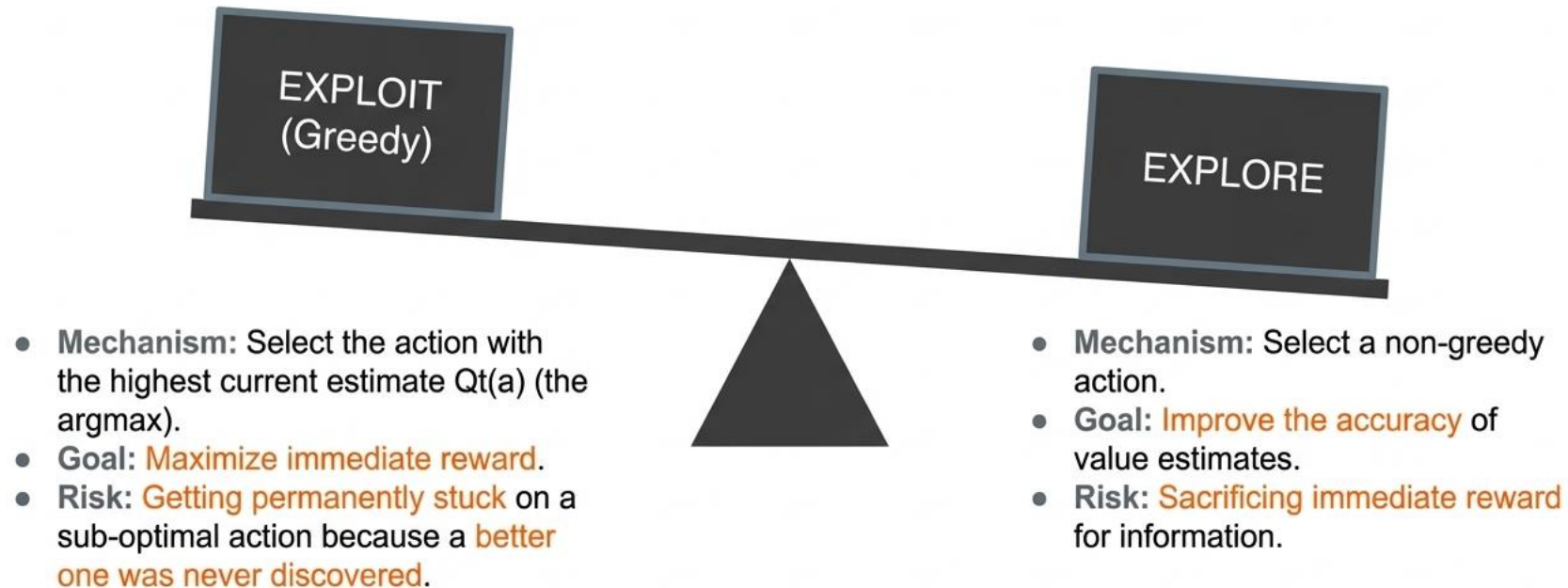
Old Estimate

Step Size
(How much to trust the new data, e.g., 1/n)

The Error
(Difference between the actual target reward and the expected value)

This recursive structure allows the agent to update its knowledge using only the most recent reward, discarding past data entirely.

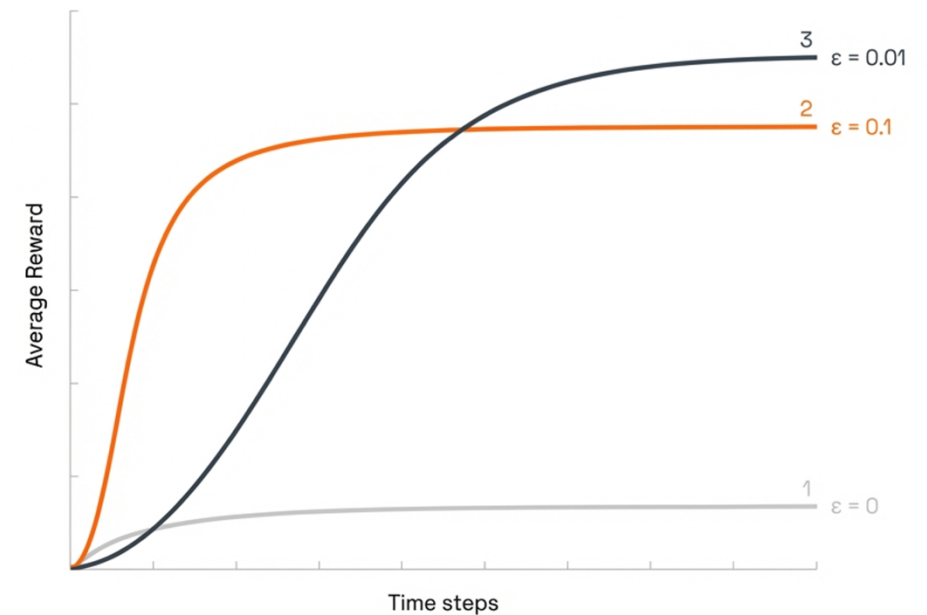
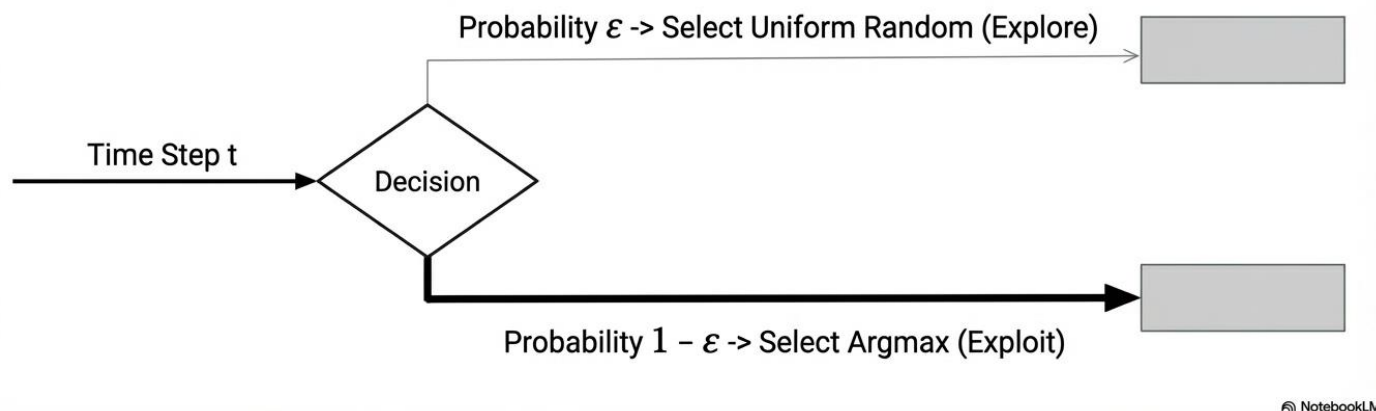
The Fundamental Dilemma



You cannot choose to do both simultaneously.

ϵ -Greedy: A Simple Mechanism for Balance

Concept: Instead of pure exploitation, the agent rolls a metaphorical die. Most of the time, it acts greedily. A small fraction of the time (ϵ), it forces exploration across all available actions to continually refine its estimates.



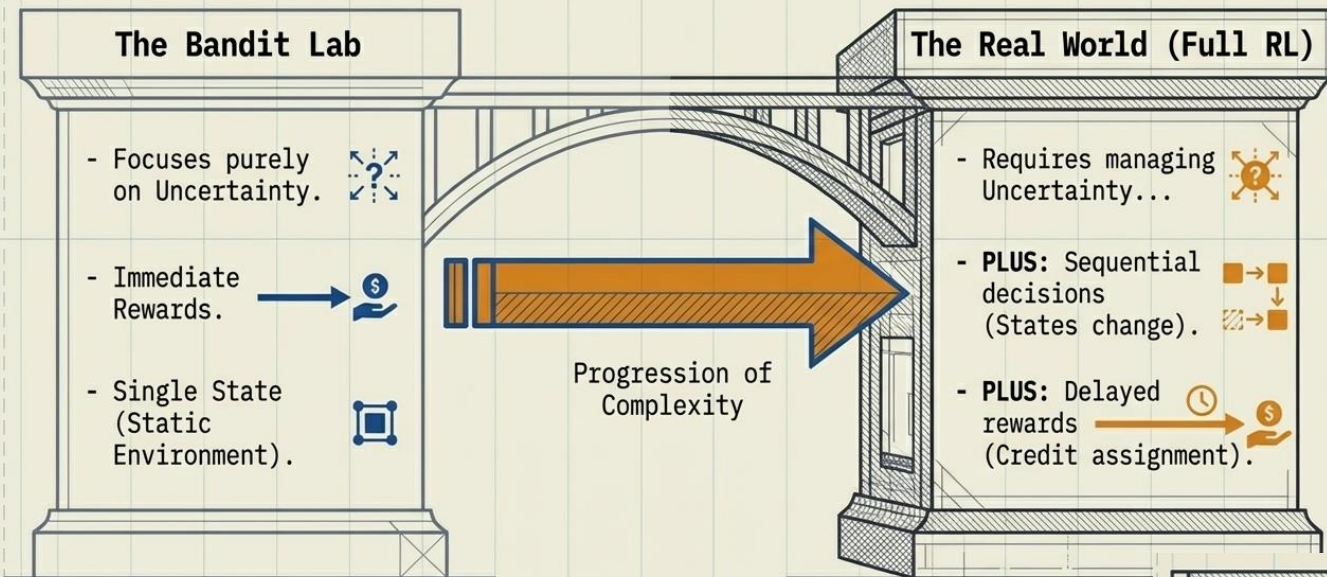
Data Insight (10-Arm Testbed):

Pure greed ($\epsilon = 0$) fails by locking onto early, lucky assumptions.

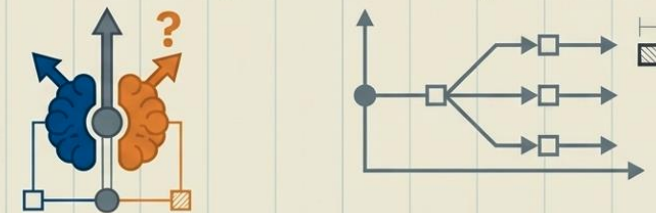
High exploration ($\epsilon = 0.1$) learns fast but permanently wastes 10% of its moves.

Low exploration ($\epsilon = 0.01$) learns slowly but ultimately achieves the highest longest long-term optimal action rate.

Transitioning to Full Reinforcement Learning



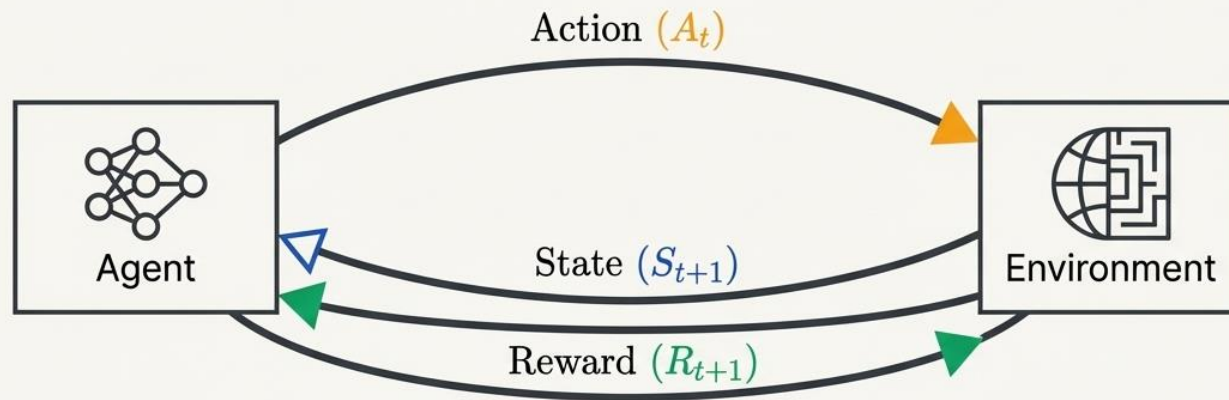
Reinforcement Learning = Learning under uncertainty + Long-term consequences



- Demo/tutorial on Bandits and Exploration/Exploitation

Markov Decision Processes

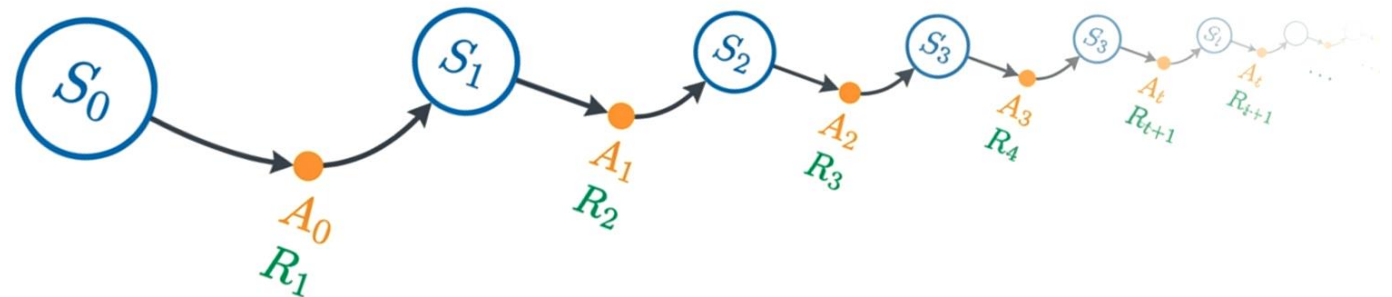
Formalizing the Agent-Environment Interface

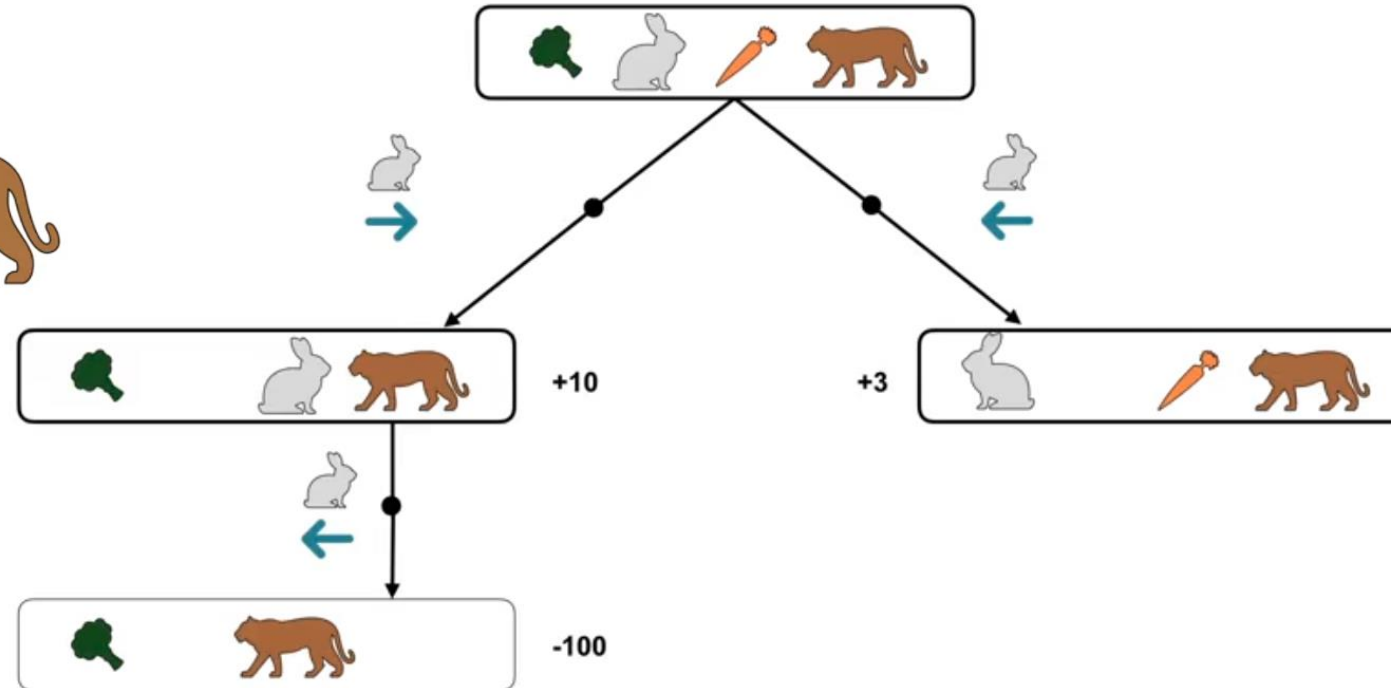
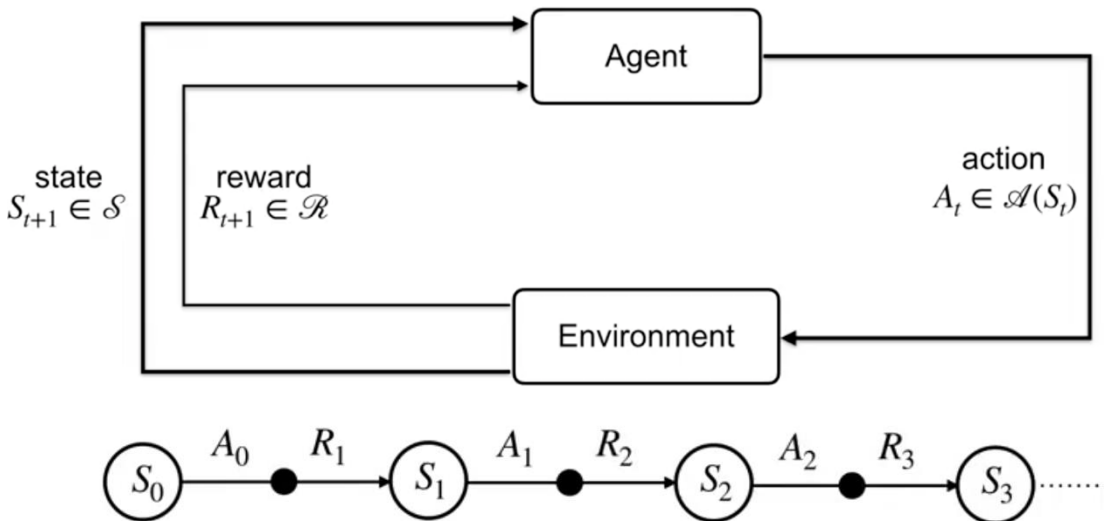
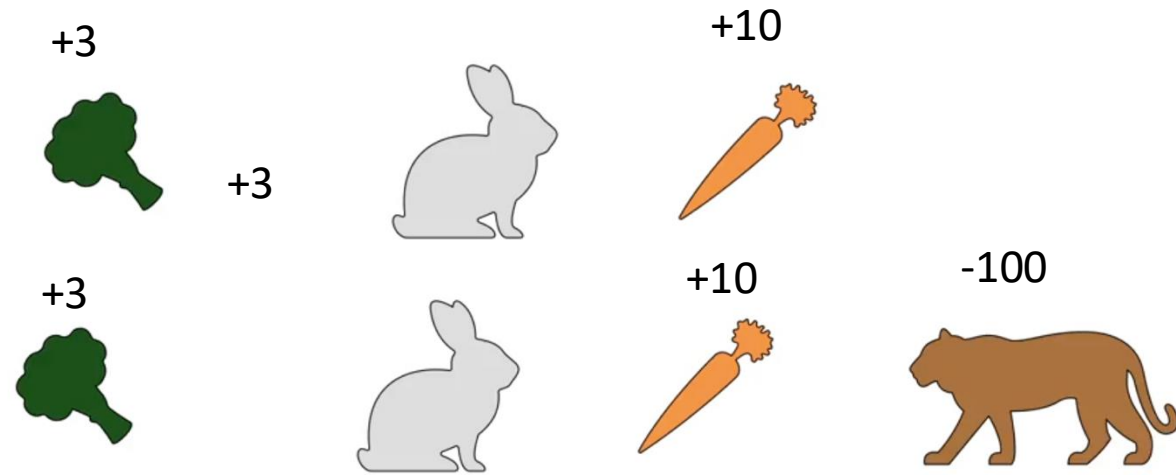


- The interaction occurs at discrete time steps: $t = 0, 1, 2, 3 \dots$
- The Trajectory: This loop generates a continuous sequence of experience:

$S_0, A_0, R_1, S_1, A_1, R_2, S_2 \dots$

- States (S_t, s)
- Actions (A_t, a)
- Rewards (R_t, r)
- Value / Returns (G_t, v_π, q_π)





The Mathematical Core of Markov Decision Processes

The 4-Element Tuple

$\mathcal{S}$: Finite set of States

$\mathcal{A}$: Finite set of Actions

$\mathcal{P}$: Transition Probabilities

$\mathcal{R}$: Expected Rewards

The Dynamics Function

$$p(s', r | s, a) = \Pr(S_t = s', R_t = r | S_{t-1} = s, A_{t-1} = a)$$

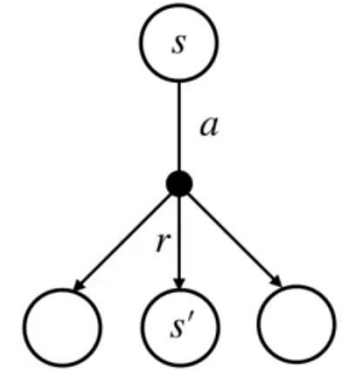
The Markov Property

The future is independent of the past given the present. The current state S_t captures all relevant information needed to predict S_{t+1} . If you know p , you have a perfect model of the world.

NotebookLM

$$p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

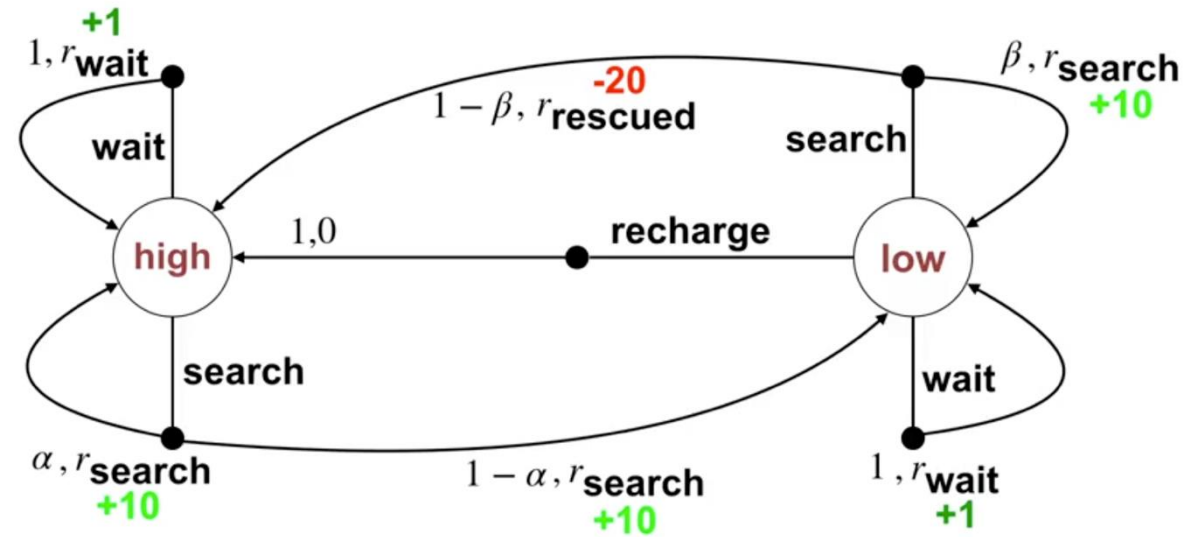


- MDPs provide a general framework for sequential decision-making
- The dynamics of an MDP are defined by a probability



$\mathcal{A}(\text{low}) = \{\text{search, wait, recharge}\}$

$\mathcal{S} = \{\text{low, high}\}$ $\mathcal{A}(\text{high}) = \{\text{search, wait}\}$



s	a	s'	r	$p(s', r s, a)$
low	wait	low	1	α
	wait	high	0	$1 - \alpha$
	search	low	10	$1 - \beta$
	search	high	-20	β
	recharge	high	0	1
	recharge	low	0	0
high	wait	high	1	1
	search	high	10	1

Exercise: Fill this table, representing the dynamics of this example.

Example: Andrew's helicopter

The helicopter's full state is given by its velocity, position, angular rate and orientation.

Observation Space: 12 dimensional, continuous valued:

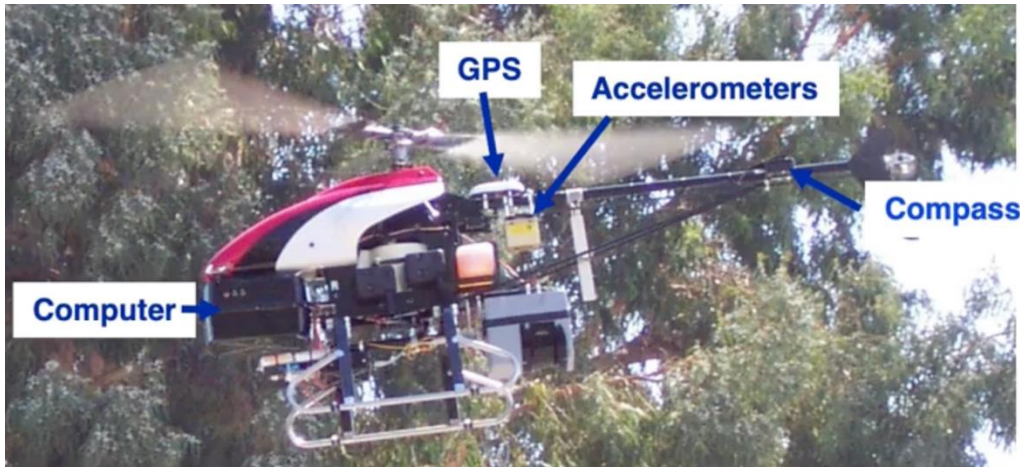
- forward velocity
- sideways velocity (to the right)
- downward velocity
- helicopter x-coord position - desired x-coord position -- helicopter's x-axis - points forward
- helicopter y-coord position - desired y-coord position -- helicopter's y-axis - points to the right
- helicopter z-coord position - desired z-coord position -- helicopter's z-axis - points down
- angular rate around helicopter's x axis
- angular rate around helicopter's y axis
- angular rate around helicopter's z axis
- quaternion x entry
- quaternion y entry
- quaternion z entry

Action Space: 4 dimensional, continuous valued:-

- longitudinal (front-back) cyclic pitch
- latitudinal (left-right) cyclic pitch
- main rotor collective pitch
- tail rotor collective pitch

Rewards: function of the 12 dimensional observation

<https://github.com/aadilh/heli-deep-q>



Goal of an Agent : Formal definition

random variable



$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

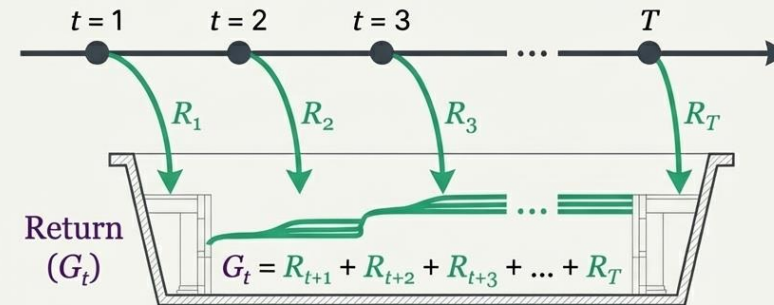
$$\mathbb{E}[G_t] = \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots]$$

maximize the expected return

Defining Success via the Reward Hypothesis

"The Reward Hypothesis":

"All goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (reward)."



Episodic Tasks

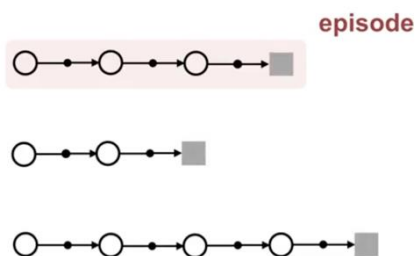
Tasks with a natural terminal state (e.g., escaping a maze, finishing a chess game).

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

Constraint: The reward signal communicates WHAT you want the agent to achieve, never HOW you want it achieved. Imparting prior knowledge via rewards leads to perverse behaviors.

© NotebookLM

Episodic Tasks



Example : Episodic Task



Every game of chess has a final move

Example : Episodes



Each game of chess is an episode

Continuing Tasks

- Interaction goes on **continually**
- No **terminal state**

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

$$= \infty?$$

How to make sure G_t is finite?

Discount the rewards in the future by γ

Where $0 \leq \gamma < 1$

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k} + \dots$$

$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Finite as long as $0 \leq \gamma < 1$

Proof

Assume R_{max} is the maximum reward the agent can receive

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \leq \sum_{k=0}^{\infty} \gamma^k R_{max} = R_{max} \sum_{k=0}^{\infty} \gamma^k$$

Finite! $\Rightarrow R_{max} \times \frac{1}{1 - \gamma}$

Effect of γ on agent behavior

$\gamma = 0$ Agent only cares about the immediate reward!
 $\Rightarrow$ Short-sighted agent!

$\gamma \rightarrow 1$ Agent takes future rewards into account more strongly
 $\Rightarrow$ Far-sighted agent!

The Recursive Engine

Expressing today's return in terms of tomorrow's return.

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Exercise 3.8 Suppose $\gamma = 0.5$ and the following sequence of rewards is received $R_1 = -1$, $R_2 = 2$, $R_3 = 6$, $R_4 = 3$, and $R_5 = 2$, with $T = 5$. What are $G_0, G_1, \dots, G_5$? Hint: Work backwards. ☐

Exercise 3.9 Suppose $\gamma = 0.9$ and the reward sequence is $R_1 = 2$ followed by an infinite sequence of 7s. What are G_1 and G_0 ? ☐

Behavior (The Policy)



$$\pi(a|s) = \Pr(A_t = a \mid S_t = s)$$

Maps States to probabilities of taking Actions.

Example: deterministic policy $\pi(s) = a$

→	→	→	🏠
→	→	↑	↑

Example: stochastic policy

100% →	→	→	🏠
50% ↑ 50% →	↑→	↑→	↑

$$\pi(a \mid s)$$

$$\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$$

$$\pi(a \mid s) \geq 0$$

Evaluation (The Value Functions)



State-Value Function

How good is it to be here?

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

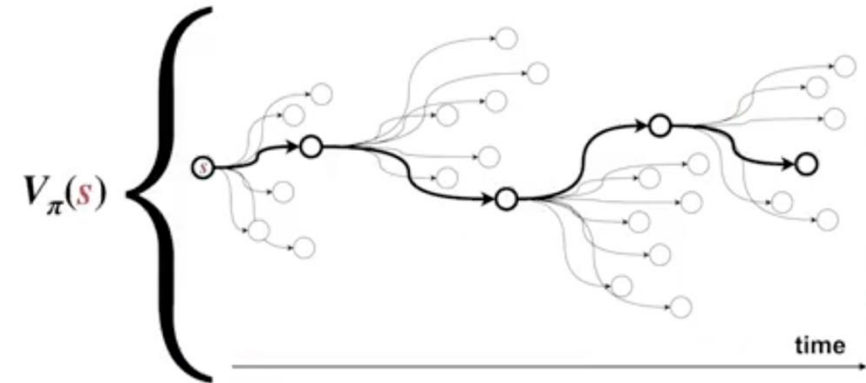
Action-Value Function

How good is it to be here AND pull this specific lever?

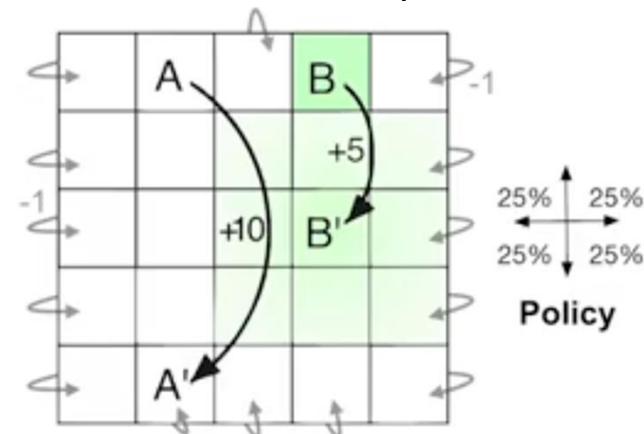
$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

NotebookLM

Value functions predict rewards into the future



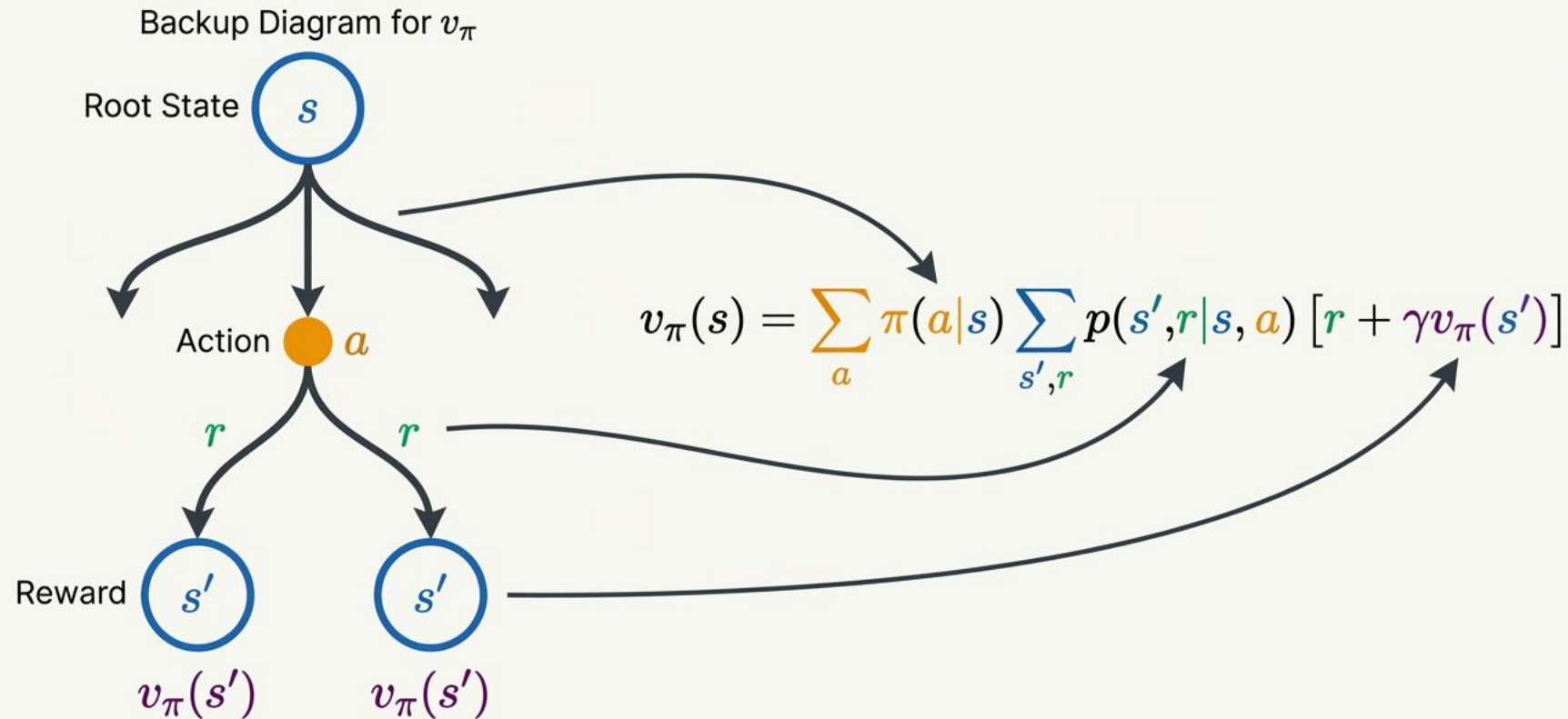
Example



$\gamma = 0.9$

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

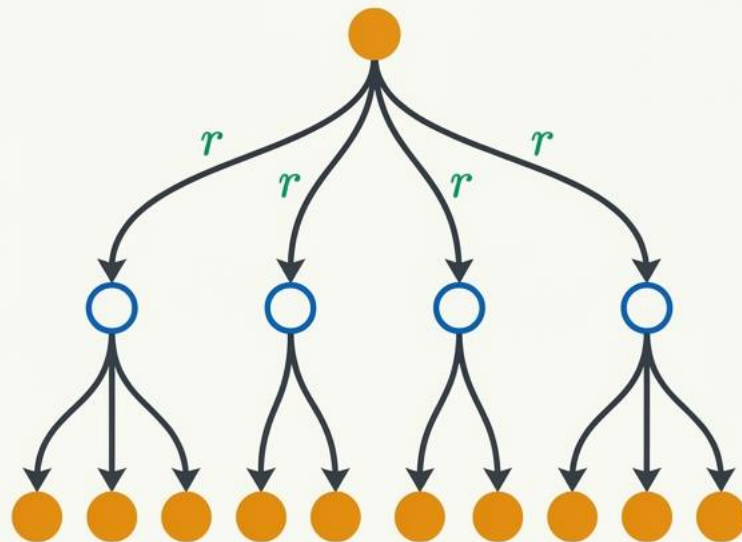
Unlocking State-Values with the Bellman Equation



Proof ?

The Action-Value Bellman Formulation

Backup Diagram for q_π



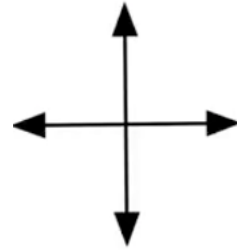
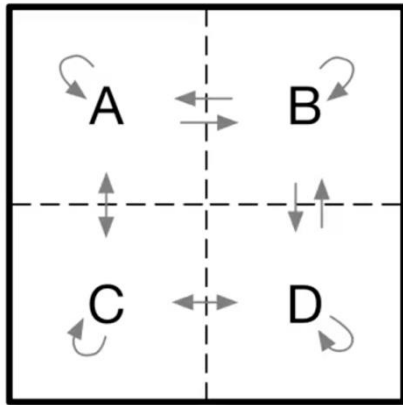
$$q_\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_\pi(s', a') \right]$$

Key Takeaway: Because the action is already chosen at the root, the agent first observes the environment's response, and then averages over its own next possible actions.

Exercise 3.12 Give an equation for v_π in terms of q_π and π .

Exercise 3.13 Give an equation for q_π in terms of v_π and the four-argument p .

Example: Gridworld



$$V_{\pi}(A) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = A]$$

$$V_{\pi}(B) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = B]$$

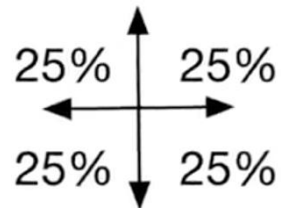
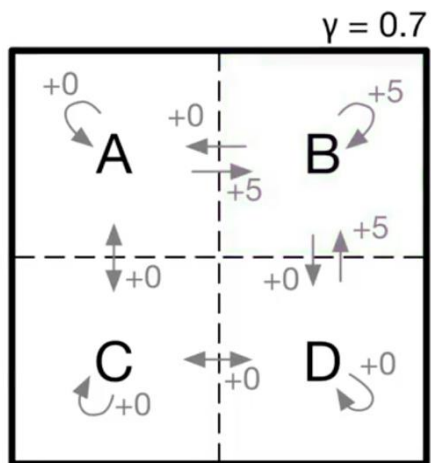
$$V_{\pi}(C) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = C]$$

$$V_{\pi}(D) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = D]$$

$$V_{\pi}(s) = \sum_a \pi(a \mid s) \sum_r \sum_{s'} p(s', r \mid s, a) [r + \gamma V_{\pi}(s')]$$

$$V_{\pi}(A) = \sum_a \pi(a \mid A) (r + 0.7 V_{\pi}(s'))$$

$$V_{\pi}(A) = \frac{1}{4}(5 + 0.7 V_{\pi}(B)) + \frac{1}{4} 0.7 V_{\pi}(C) + \frac{1}{2} 0.7 V_{\pi}(A)$$



$$V_{\pi}(A) = \frac{1}{4}(5 + 0.7 V_{\pi}(B)) + \frac{1}{4} 0.7 V_{\pi}(C) + \frac{1}{2} 0.7 V_{\pi}(A)$$

$$V_{\pi}(B) = \frac{1}{2}(5 + 0.7 V_{\pi}(B)) + \frac{1}{4} 0.7 V_{\pi}(A) + \frac{1}{4} 0.7 V_{\pi}(D)$$

$$V_{\pi}(C) = \frac{1}{4} 0.7 V_{\pi}(A) + \frac{1}{4} 0.7 V_{\pi}(D) + \frac{1}{2} 0.7 V_{\pi}(C)$$

$$V_{\pi}(D) = \frac{1}{4}(5 + 0.7 V_{\pi}(B)) + \frac{1}{4} 0.7 V_{\pi}(C) + \frac{1}{2} 0.7 V_{\pi}(D)$$

$$V_{\pi}(A) = 4.2$$

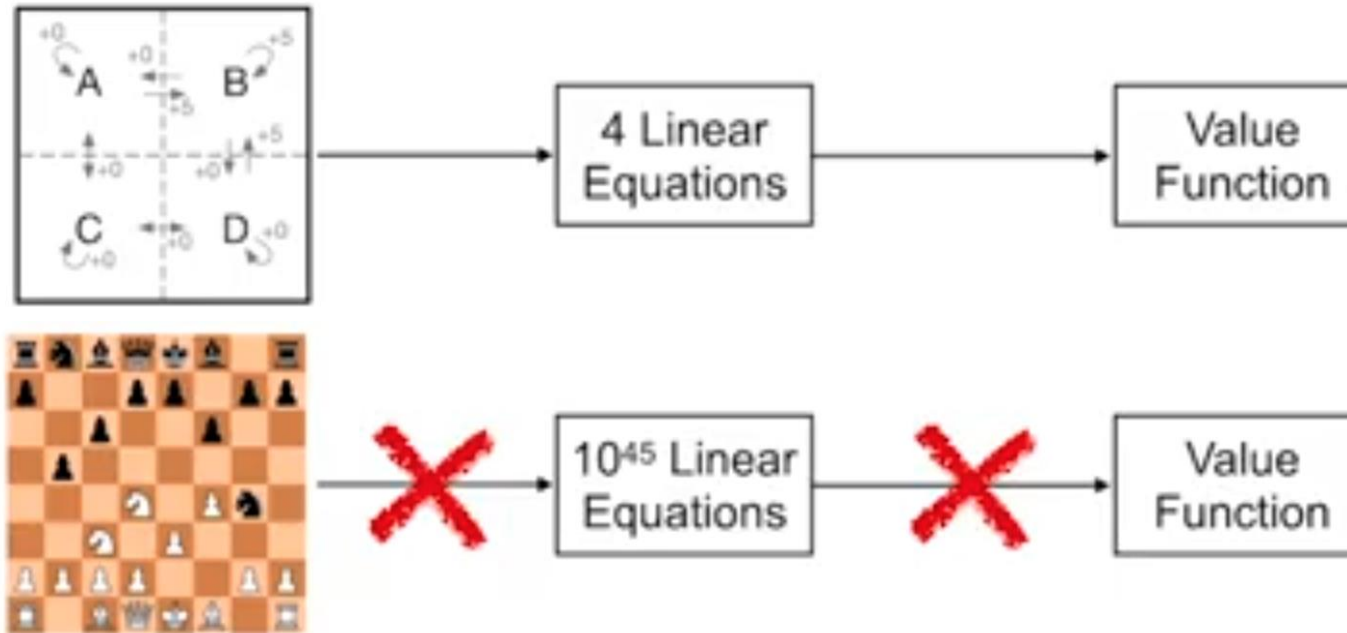
$$V_{\pi}(B) = 6.1$$

$$V_{\pi}(C) = 2.2$$

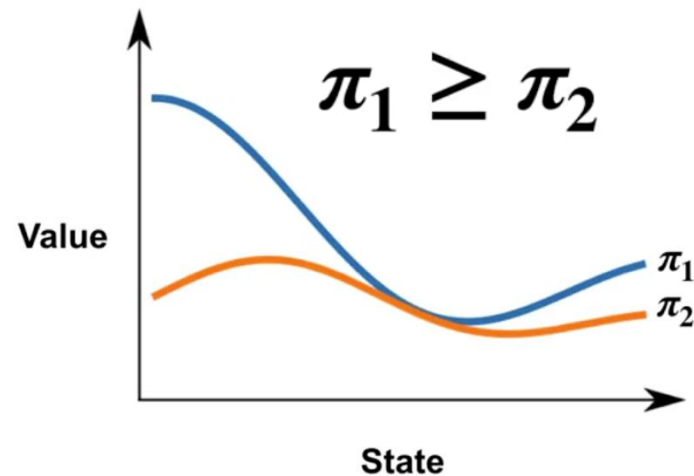
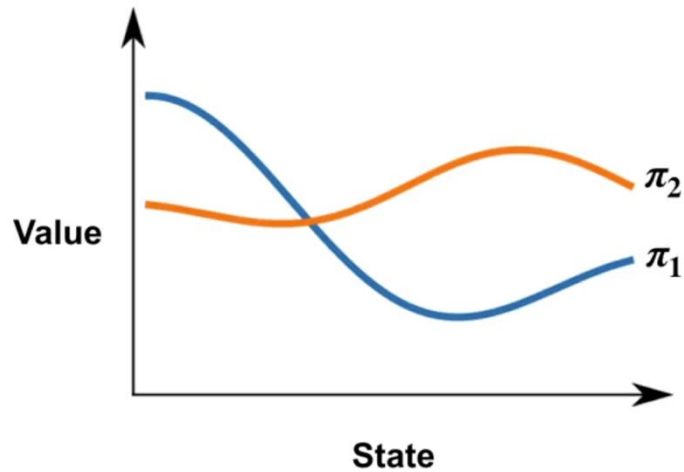
$$V_{\pi}(D) = 4.2$$



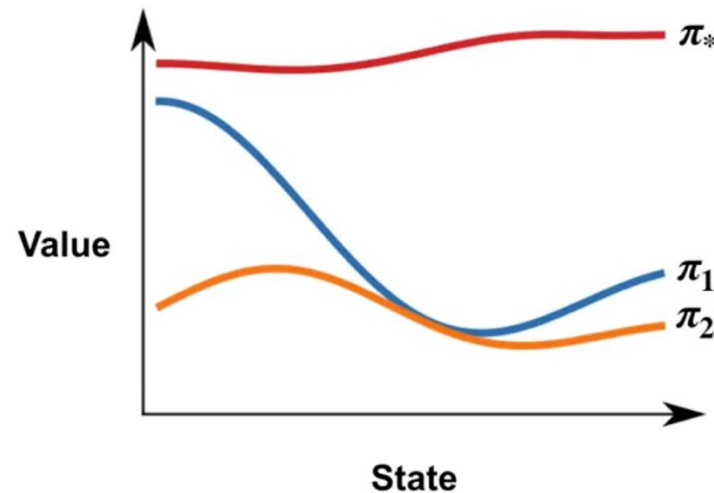
We can only directly solve small MDPs



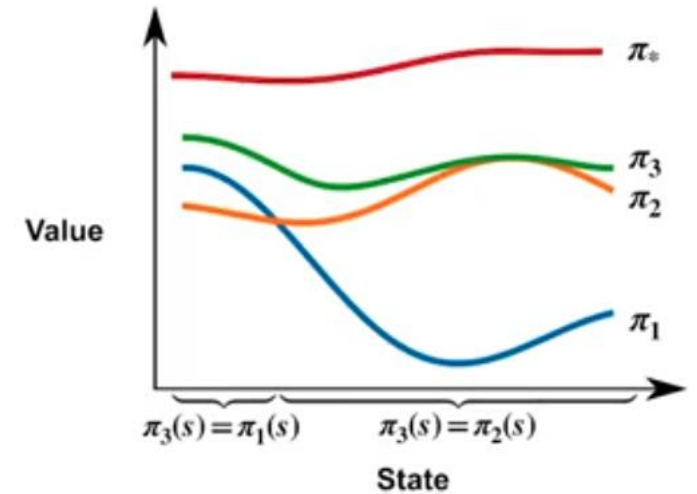
- You can use the **Bellman Equations** to solve for a **value function** by writing a **system of linear equations**
- We can only solve **small MDPs** directly, but **Bellman Equations** will factor into the solutions we see later for **large MDPs**



An **optimal policy** π_* is as good as or better than all the other policies



There is always an optimal policy



There's a rigorous proof showing that there must always exist at least one optimal deterministic policy.

Defining the Ultimate Goal of Optimality

Partial Ordering Constraint:

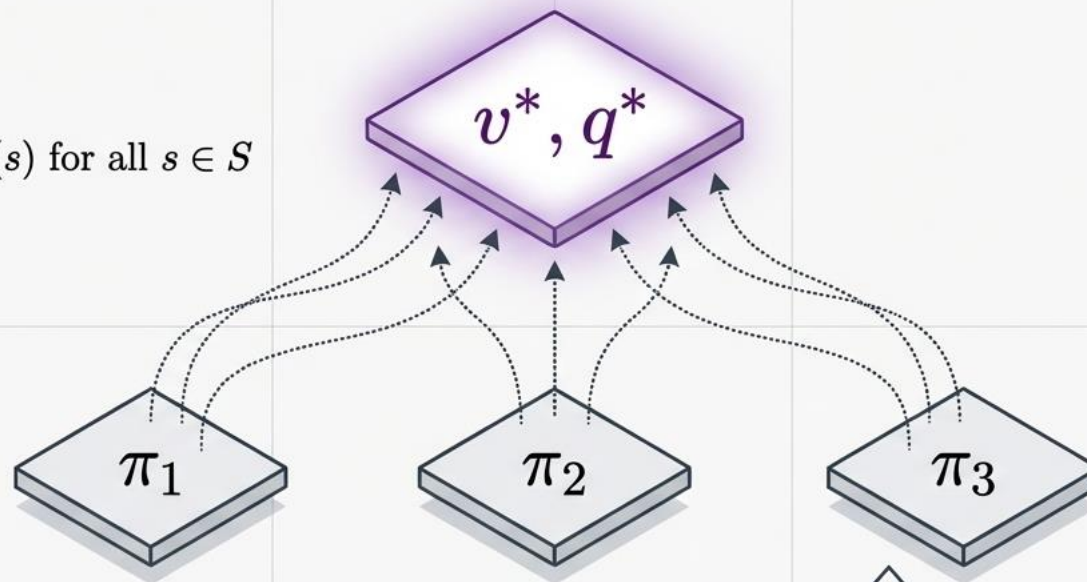
$\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in S$

Optimal State-Value:

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

Optimal Action-Value:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$



Crucial Distinction:

There can be many optimal policies (e.g., multiple equally fast paths through a maze), but they all mathematically **share exactly one optimal value function**.

Recall that

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

$$v_{*}(s) = \sum_a \pi_{*}(a|s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_{*}(s')]$$

$$v_{*}(s) = \max_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_{*}(s')]$$



Bellman Optimality Equation for v_{*}

Recall that

$$q_{\pi}(s, a) = \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right]$$

$$q_{*}(s, a) = \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi_{*}(a' | s') q_{*}(s', a') \right]$$

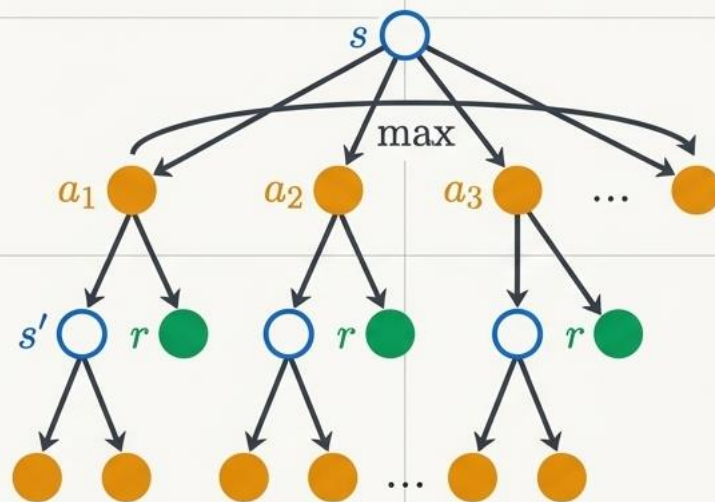
$$q_{*}(s, a) = \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \max_{a'} q_{*}(s', a') \right]$$



Bellman Optimality Equation for q_{*}

The Dynamic Schematic

Solving for Maximum Return: Bellman Optimality

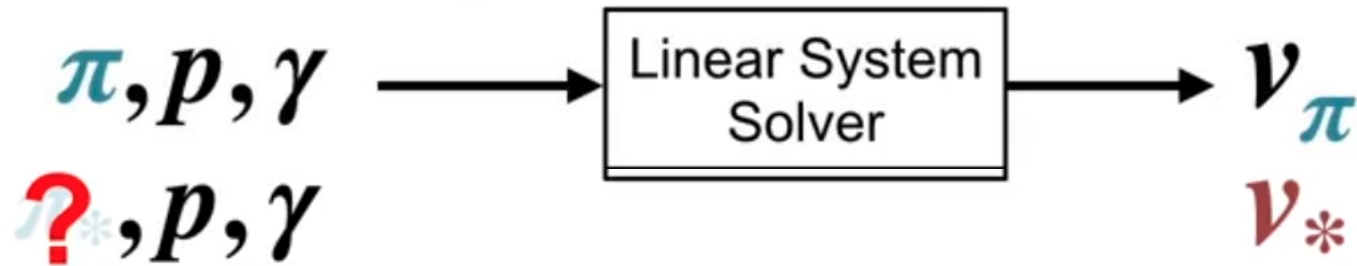


$$v^*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^*(s')]$$

$$q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q^*(s', a')]$$

Takeaway: An optimal agent does not average its outcomes over bad actions; it exclusively and deterministically picks the highest-yielding action.

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

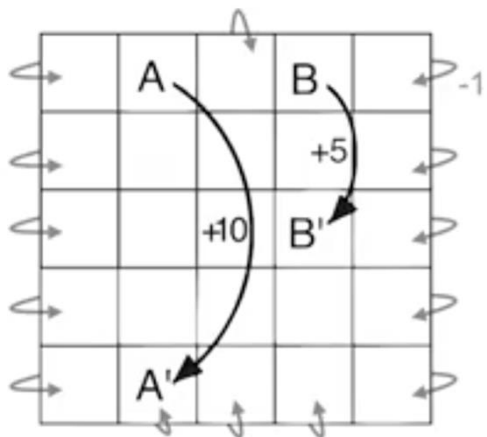


Not linear

$$v_*(s) = \max_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$



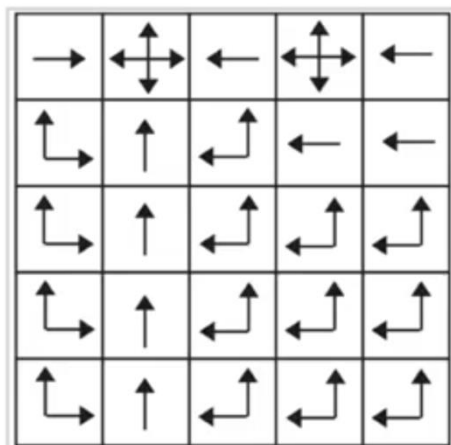
Determining an Optimal Policy



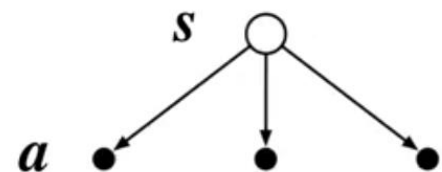
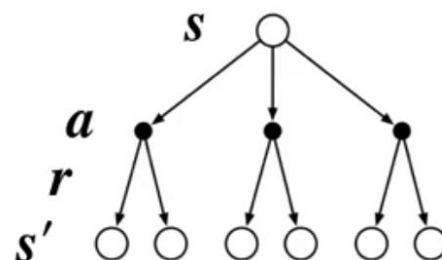
$\gamma = 0.9$

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

v_*

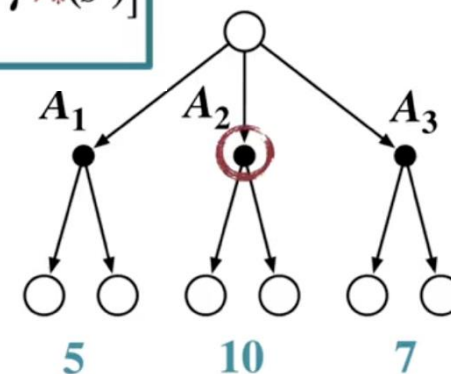


π_*



$$v_*(s) = \max_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$

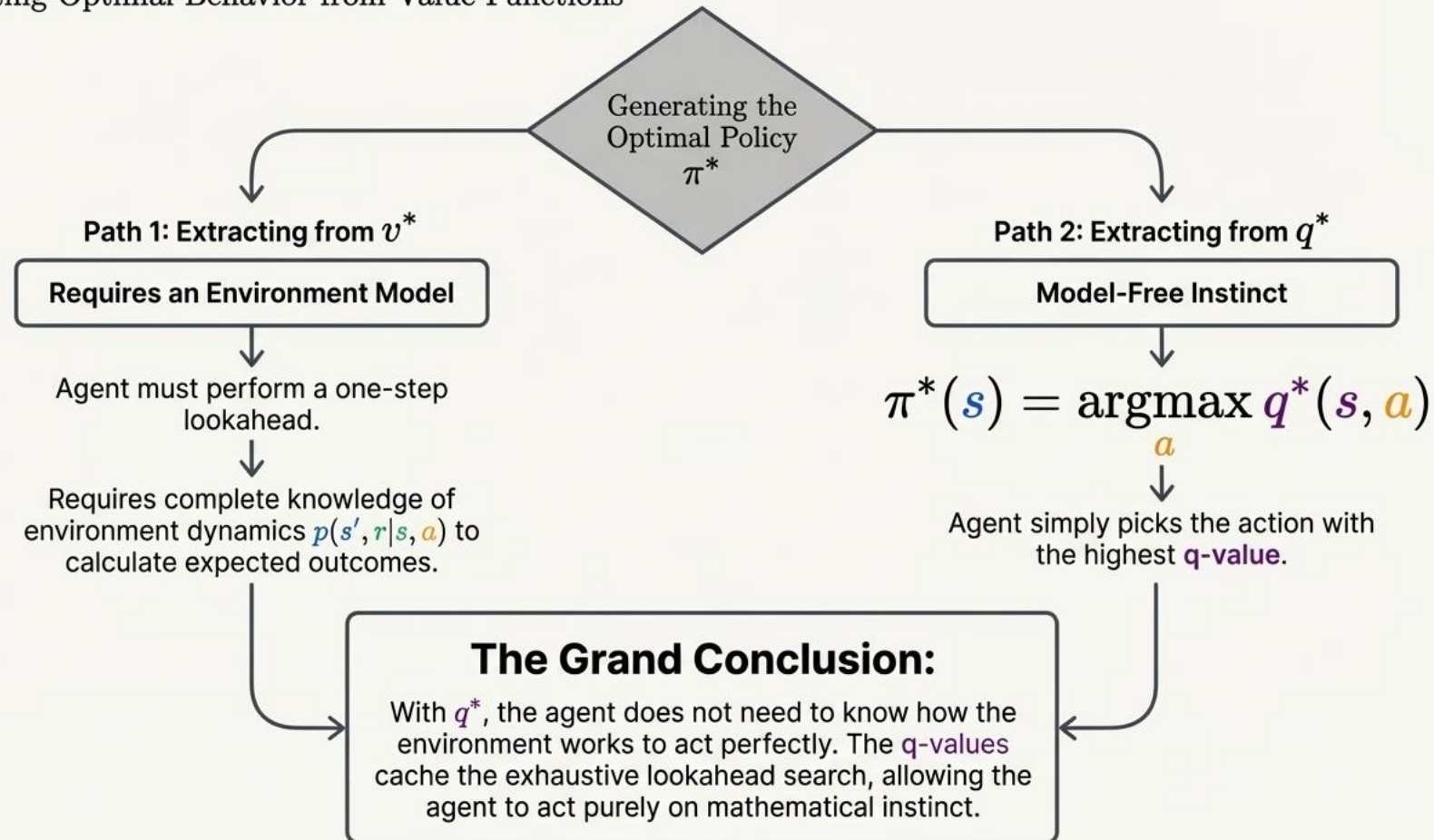
$$\pi_*(s) = \operatorname{argmax}_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$



$$\pi_*(s) = \operatorname{argmax}_a \sum_{s'} \sum_r p(s', r | s, a) [r + \gamma v_*(s')]$$

$$\pi_*(s) = \operatorname{argmax}_a q_*(s, a)$$

Extracting Optimal Behavior from Value Functions



- Remark: In this example, we know exactly how the environment behaves, so we can compute the best action analytically.
- In reinforcement learning, we don't know this — we must learn it from experience.
- **In RL (later):** Model is unknown $\rightarrow$ we replace:
 - $\sum p(\cdot)$ with samples (r, s')
- This leads to: TD learning, Q-learning

- [1] Sutton, R. S. and Barto, A. G. (2018), Reinforcement Learning: An Introduction (Second edition). MIT Press
- [2] Adam White and Martha White, Reinforcement Learning Specialization, University of Alberta, Coursera